HandSense: informe de avances
Prototipo experimental de monitoreo de pinza

Alex Díaz Canales María José Córdova

10 de noviembre de 2026

Cómo usar este informe

Escribimos esta guía para quien recién comienza con microcontroladores. Primero explicamos qué queremos observar; después seguimos el recorrido de la señal y probamos cada bloque. En cada paso indicamos qué hacer, por qué lo hacemos y qué debería ocurrir. Las instrucciones reproducibles de hoy corresponden a Wokwi y al código del repositorio. El montaje físico se completará cuando confirmemos los modelos recibidos y sus mediciones.

Estado de compras informado por Alex: ESP32 y sensores comprados por AliExpress; recepción y modelos exactos por registrar. Planeamos comprar componentes complementarios en San Diego. Comprar una pieza no significa que ya esté recibida ni probada.

Recorrido: conceptos del ramo, sensores, simulación, programa, pruebas, montaje físico y calibración. Después presentamos avances, límites y próximos pasos.

1. Lo que ya conocemos del ramo

En Desafío PIC seguimos una cadena: sensor, voltaje, ADC, programa y display. HandSense conserva esa lógica y añade el historial del movimiento: una sola lectura no basta para reconocer una repetición completa.

Tema del curso	Aplicación en HandSense
Variables y constantes (semana 1)	Una variable guarda la lectura actual; una constante identifica un pin o intervalo.
Condicionales (semana 2)	Un `if` decide si la señal pasó un umbral.
Ciclos (semanas 2–3)	El PIC repite `while(1)`; el entorno Arduino llama repetidamente a `loop()`.
Arreglos y funciones (semana 4)	Un arreglo conserva cinco lecturas. Una función encapsula una tarea para revisarla por separado.
Puertos y configuración (semanas 7–8)	En PIC usamos TRIS/PORT; en Arduino-ESP32 usamos `pinMode` y `digitalWrite`. Las direcciones de registros y los pines cambian.
ADC (semana 9, tutorial 4)	Convertimos tensión en un número. El ADC no reconoce por sí mismo una pinza.
Comunicación serie (semana 9, tutorial 5)	El monitor serie permite observar lecturas y decisiones; más adelante la app recibirá información por Wi-Fi.

Estas conexiones se basan en los materiales del ramo disponibles en la carpeta clases. El firmware de HandSense utiliza C++ con el entorno Arduino: conserva estructuras de C vistas en clases y añade clases como `GestureDetector`. No se compila con XC8 ni utiliza el archivo HEX del PIC.

2. Paso 1: entender qué mide cada pieza

El sensor flexivo cambia su resistencia al curvarse. El FSR cambia su resistencia cuando recibe fuerza. El microcontrolador lee tensión, por lo que necesitamos un divisor: sensor y resistencia fija conectados en serie. El punto entre ambos va al ADC.

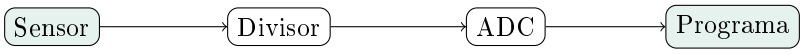


Diagrama propio, 10 de noviembre de 2026: recorrido conceptual, no esquema de cableado final.

Si el sensor está arriba y la resistencia fija abajo, la relación ideal es

$$V_{ADC} = V_{alimentacion} \frac{R_{fija}}{R_{sensor} + R_{fija}}.$$

Invertir las posiciones invierte el sentido del cambio. Por eso mediremos primero qué lectura corresponde a reposo y cuál a flexión/contacto. La resistencia se elegirá a partir del sensor recibido. Compartir GND permite que todos los bloques usen la misma referencia de tensión. No trasladaremos la alimentación de 5 V del PIC a las entradas del ESP32.

Antes de seguir: Poder explicar qué cambia en cada sensor y por qué hace falta convertir resistencia en tensión.

3. Paso 2: reproducir la simulación actual

Se necesita acceso a Wokwi y a la carpeta `firmware/logica_gesto/` del proyecto. La simulación usa ESP32 DevKit clásico; sus pines no deben copiarse a otra placa sin revisar el pinout.

1. Crear un proyecto ESP32 en Wokwi y copiar los tres archivos de esa carpeta: `sketch.ino`, `gesture_logic.h` y `diagram.json`. El archivo JSON describe el circuito existente.
2. Comprobar que hay dos potenciómetros. Sus extremos están en 3V3 y GND; la salida SIG del primero va a D34 y la del segundo a D35. El potenciómetro sustituye temporalmente al sensor y entrega una tensión controlable.
3. Verificar los LED: D18 al ánodo verde; su cátodo llega a GND mediante 220 ohmios. D19 se conecta de igual manera al LED naranja. Son los valores del circuito simulado, no una selección definitiva para el hardware.
4. Iniciar con ambas perillas al mínimo. El programa abre comunicación serie a 115200 baudios. Esperar al menos 100 ms para llenar el filtro.
5. Girar solo la primera perilla y observar F; después regresar a cero y girar solo la segunda para observar C.

Antes de seguir: Cada perilla debe modificar su canal. Guardar las lecturas y una captura real; no continuar si los canales están intercambiados.

4. Paso 3: leer el programa sin perderse

`setup()` configura una vez; `loop()` revisa continuamente el tiempo y las entradas. Este fragmento procede del sketch:

```
Serial.begin(115200);
analogReadResolution(12);
pinMode(FLEX_PIN, INPUT);
pinMode(VALID_LED_PIN, OUTPUT);
```

Con 12 bits hay $2^{12} = 4096$ códigos, de 0 a 4095 [1]. Más códigos no aseguran más exactitud. La tensión de fondo de escala depende de la configuración del ADC; no debemos asumir que cada código equivale exactamente a 3,3/4095 voltios.

El programa toma una muestra cada 20 ms, unas 50 por segundo. Consulta `millis()` para saber cuánto tiempo pasó, de modo que puede seguir atendiendo otras tareas. Una media de las últimas cinco muestras suaviza cambios rápidos. Ese filtro también retrasa la respuesta; por eso se debe comprobar con movimientos reales.

Antes de seguir: Identificar en el sketch las constantes de pines, intervalo de lectura y número de muestras del filtro.

5. Paso 4: reconocer un movimiento completo

La histéresis usa dos límites: uno para entrar en un estado y otro para salir. En Wokwi entramos desde 2457 y salimos en 1433; son valores provisionales. Esta separación evita encender y apagar un estado por pequeñas oscilaciones alrededor de un solo límite.

Una máquina de estados recuerda lo que ocurrió. REPOSO significa que ninguna entrada está activa; FLEXION indica flexión; CONTACTO indica contacto; LIBERACION describe la espera de retorno cuando el contacto ya se soltó. ESPERA_REPOSO bloquea un intento que agotó el tiempo hasta que ambas entradas bajan. La regla guarda si hubo flexión y contacto y cuenta cuando ambas entradas vuelven a reposo.

1. Subir flexión y luego contacto; bajar contacto y flexión: esperamos `fusion=1`, `flex_solo=1`, `inc=0` partiendo de cero.

2. Reiniciar; subir y bajar solo flexión: esperamos 0, 1 y 1, con evento SIN_CONTACTO.

3. Reiniciar; subir y bajar solo contacto: esperamos 0, 0 y 1, con evento SIN_FLEXION.

4. Reiniciar; subir ambas y mantener más de tres segundos: esperamos SIN_LIBERACION y ningún conteo completo. Bajar ambas para habilitar otro intento.

5. Reiniciar y hacer dos pinzas completas: esperamos dos repeticiones, sin incremento extra mientras se mantiene una posición.

Los eventos son breves: guardar el registro serie permite revisarlos. El LED naranja indica intento incompleto; no prueba la vibración física. Los tres segundos son un parámetro de prueba, no una recomendación para ejercitarse.

Antes de seguir: Anotar resultado esperado y observado en ensayos.csv; una simulación correcta prueba la lógica bajo esas entradas.

6. Paso 5: pasar a los componentes comprados

Al recibir el pedido anotaremos modelo exacto, fotos y ficha técnica. Si es un XIAO ESP32-C3, D34/D35 no existen y el pinout se debe adaptar. Primero probaremos cada sensor en mesa y después el motor con su etapa de control. La vista física final se añadirá con conexiones verificadas.

Para San Diego revisaremos qué falta de protoboard, cables flexibles, conectores, resistencias, LED, interruptor, aislación y materiales de sujeción. El motor, transistor/MOSFET y protección se seleccionarán juntos según tensión y corriente; conocer solo el nombre ESP32 no basta para escogerlos. Registraremos también el costo de envío y materiales, además del precio de los sensores.

Antes de seguir: Tener modelos y recepción registrados antes de copiar conexiones de un dibujo de internet.

7. Paso 6: calibrar y comunicar lo observado

Calibrar significa relacionar lecturas con estados conocidos de este montaje: reposo y un gesto cómodo. Guardaremos señales de desarrollo para decidir polaridad, rango e histéresis. Después fijaremos los parámetros y evaluaremos ensayos nuevos; ajustar usando los mismos ensayos de evaluación escondería errores.

La memoria no volátil conservará parámetros al apagar; las variables normales en RAM no cumplen esa función. En ESP32 se puede usar Preferences/NVS [2], vinculando el objetivo de EEPROM del ramo con otra implementación. Todavía falta implementar y verificar esa persistencia.

El monitor serie ayuda a depurar. El dashboard será la interfaz para quien use el dispositivo: debe mostrar datos enviados por el ESP32. Hoy la app muestra ejemplos simulados. Antes de integrar se debe resolver que ESPERA_REPOSO no está admitido por su validación. Una pantalla que dice señal inválida no demuestra que el firmware detecte realmente esa condición.

Antes de seguir: Al reiniciar deben recuperarse los parámetros; al mover una entrada física, app y monitor serie deben concordar. Ambas pruebas siguen pendientes.

8. Problema y alcance

Nuestro objetivo es observar una pinza pulgar–índice mediante flexión y contacto relativos, contar repeticiones y explicar intentos incompletos. Construiremos un prototipo con dos entradas analógicas, LED, motor vibrador, memoria no volátil y dashboard inalámbrico. La propuesta es experimental: aún no hemos demostrado utilidad clínica ni uso doméstico independiente.

9. Antecedentes y aporte propuesto

SensorE ya combina medición de flexión y fuerza en un wearable [4]. Por eso nuestro aporte propuesto se centra en una tarea explícita, conteo interpretable y evaluación del costo y facilidad de colocación. RehabBuddy muestra que la calibración asistida y los problemas técnicos limitan conclusiones sobre uso independiente [5]. Estas fuentes orientan decisiones; sus resultados no son resultados de HandSense.

10. Arquitectura y componentes

Dos sensores resistivos se conectarán mediante divisores a entradas ADC. El ESP32 procesará las señales y manejará LED, driver de motor y comunicación. La app mostrará estados y lecturas relativas. La placa y los sensores están comprados por AliExpress según Alex; los modelos exactos, recepción, resistencias, corriente del motor y costo final siguen por registrar; la tabla BOM del repositorio registra estos campos sin inventar precios. La guía FSR indica dependencia de la geometría y distribución de fuerza [6].

11. Estado observado

Bloque	Evidencia y pendiente
Adquisición	Potenciómetros en Wokwi; faltan sensores físicos.
Algoritmo	Cinco pruebas locales; falta evaluación física etiquetada.
App	Datos simulados y contrato HTTP; falta enlace real.
Integración	ESPERA_REPOSO existe en firmware pero no en la validación de la app.
Memoria y motor	Pendientes de implementación y prueba física.
Soporte	Pendiente de prueba de colocación y comodidad.

12. Calibración y método experimental

Registraremos reposo y gestos de desarrollo para determinar polaridad, rango y ruido. Los parámetros se fijarán antes de evaluar nuevos ensayos. Compararemos conteo por flexión y por fusión sobre las mismas señales, con etiqueta manual. Informaremos falsos positivos, falsos negativos y tamaño real de muestra. Repetiremos colocación y verificaremos persistencia tras reinicio. El protocolo detallado y la plantilla vacía están en validacion/.

13. Resultados y limitaciones

Las pruebas locales verifican reglas de software. No prueban sensores, electrónica, conexión inalámbrica, comodidad ni efectos terapéuticos. Las mediciones físicas, gráficos y fotografías quedan pendientes; se incorporarán con fecha, montaje y versión del código. No se incluyen fotografías reconstruidas ni datos sintéticos como resultados físicos.

14. Planificación y presupuesto

La Gantt operativa está en SEGUIMIENTO_NOVIEMBRE.md. Se conserva como antecedente la carta de septiembre. El syllabus ubica la evaluación en la semana del 23 de noviembre; el día 25 está por confirmar. La meta interna es estar listos el 23. Alex lidera software y pruebas; Cote lidera materiales y soporte, sujeto a acuerdo del equipo. La integración y defensa se realizan juntos. El presupuesto final depende de las piezas confirmadas, envío, materiales y reposición.

15. Accesibilidad y futuro

Mediremos colocación, ayuda necesaria, comprensión de mensajes y percepción de vibración. El costo bajo y la facilidad de uso son hipótesis que deben verificarse. Después de la entrega proponemos co-diseño con usuarios y terapeutas, pruebas de confiabilidad y análisis de mantenimiento. Usuario, comprador y pagador podrían ser distintos; no contamos aún con validación de demanda ni interés inversor documentado.

16. Licencia y posibilidades de reutilización

En la revisión del repositorio del 10 de noviembre no encontramos un archivo de licencia general ni uno específico de HandSense. La guía está escrita para facilitar su comprensión, pero la licencia de reutilización todavía debe ser acordada por los autores. Los materiales de clases y artículos citados conservan sus propios derechos; no se incorporan como contenido propio.

Referencias

[1] Espressif. Arduino-ESP32 ADC API. Consultado el 10 de noviembre de 2026. https://docs.espressif.com/projects/arduino-esp32/en/latest/api/adc.html.

[2] Espressif. Preferences API. https://docs.espressif.com/projects/arduino-esp32/en/latest/api/preferences.html.

[3] Materiales de Microcontroladores: semanas 1–4 (C, decisiones, ciclos y funciones); semana 7 (PIC y primer programa); semana 8 (configuración); semana 9 (ADC y comunicación serie). Archivos proporcionados por Alex en Documents/Microcontroladores/clases. No se redistribuyen los PDF del curso.

[4] Bates et al. (2026). Multimodal measurement of hand function with the SensorE Exoskeleton. https://doi.org/10.1088/2057-1976/ae3e98.

[5] A feasibility study to empower patients with individualised physical rehabilitation through wearable technology: RehabBuddy (2026). https://pubmed.ncbi.nlm.nih.gov/42964283/.

[6] Interlink Electronics. FSR Integration Guide. https://www.interlinkelectronics.com/downloads/integration-guides/fsr-400-series-integration-guide.pdf.